

BRIEF TÉCNICO

Módulo de Gestão Financeira & FinOps

Plataforma de FinOps para operações de IA — rastreamento de tokens, P&L; por cliente, billing B2B2C, degradação ativa e integração com L2 ATLAS.

Versão	1.0
Data	Julho 2026
Status	Em produção
Stack	Next.js 16 · React 19 · TypeScript 5 · SQLite/Supabase
Equipe	Davi + Artur (L2 Systems)
Classificação	CONFIDENCIAL — uso interno e colaboradores autorizados

1. O QUE É

O **L2 Cashflow** é o módulo de gestão financeira e FinOps da L2 Systems. Ele começou como um fluxo de caixa simples entre sócios e evoluiu para uma **plataforma de FinOps para operações de IA** — o tipo de sistema que uma empresa de tecnologia precisa para controlar quanto gasta com modelos de linguagem, quanto fatura por cliente, e quanto lucro sobra no final do mês.

Hoje ele opera em produção dentro do ecossistema L2 ATLAS, integrado via protocolo MCP (Model Context Protocol) e webhooks em tempo real.

2. O QUE JÁ FAZ (Estado Atual)

2.1 Gestão Financeira Base

- **Clientes** — cadastro, contratos mensais, controle de pagamento recorrente, alertas de vencimento
- **Faturas** — emissão, acompanhamento (pendente, pago, atrasado), automação de status
- **Despesas** — categorizadas (Software, Marketing, Equipamento, Infraestrutura, Pessoal, Outros), recorrentes ou avulsas, vinculadas ou não a cliente
- **Carteira de Sócios** — injeções e retiradas de Artur e Davi com saldo atualizado em tempo real
- **Relatórios** — dashboard consolidado com gráficos de gastos, heatmap de uso e feed de atividades

2.2 FinOps (Operações de IA)

- **Rastreamento de tokens** — cada chamada a um modelo de IA é registrada com input tokens, output tokens, cache hit/miss, custo USD e BRL
- **Rate cards por modelo** — tabela com preço por 1M tokens input/output/cache, contexto máximo, flags de suporte (tools, caching, JSON)
- **Custo normalizado** — cálculo exato de cada evento usando rate cards do banco, com fallback se o modelo não estiver catalogado
- **Degradação ativa** — motor que monitora gasto por usuário e dispara webhooks para rebaixar modelos quando limites são excedidos (warning R\$25 / hard cap R\$35)
- **Forecasting** — projeção de custo mensal baseada na média diária, com alertas verde/amarelo/vermelho

2.3 Billing B2B2C (Plus)

- Assinaturas premium de usuários finais com split de receita (gateway ~5%, L2 ~30%, cliente ~70%)
- Gateways suportados: Stripe, Hotmart
- Eventos de faturamento com rastreamento completo de taxas e repasses

2.4 Relatórios Enterprise

- **P&L por cliente** — receita contratada vs custo de IA vs margem
- **Cost Explorer** — custo por modelo, top 10 usuários por gasto, taxa de cache hit
- **Forecast** — projeção de fim de mês + simulador de margem
- **Relatório Comercial** — receita total + Plus, margem bruta e líquida
- **Relatório Operacional** — sessões, tokens, custo médio por sessão, breakdown por modelo

3. COMO FUNCIONA (Arquitetura)

3.1 Stack Tecnológica

Camada	Tecnologia	Versão
Runtime	Node.js	20+
Framework	Next.js (App Router)	16.1.6
UI	React + Tailwind CSS 4 + Framer Motion	19.2.3
Banco Local	SQLite (better-sqlite3)	12.6.2
Banco Cloud	PostgreSQL (Supabase)	—
Ícones	Lucide React	0.575
Gráficos	Recharts	3.7
Exportação	jsPDF + docx	—
MCP SDK	@modelcontextprotocol/sdk	1.29

Dependências adicionais: *clsx, tailwind-merge, jspdf-autotable, @opengsd/gsd-core.*

3.2 Camadas da Aplicação

Camada	Onde	Função
Apresentação	app/*/page.tsx + components/	Páginas React com SSR/CSR
Servidor	app/actions.ts	Server Actions com revalidatePath
Dados	lib/repositories/	Interface + implementações SQLite/Supabase
Banco	lib/db/ + supabase/	Schema SQLite + RPC functions PostgreSQL

3.3 Dual Backend (SQLite ↔ Supabase)

O sistema opera com dois backends intercambiáveis. A seleção é automática via variável de ambiente:

ATLAS_CASHFLOW_DB=local	Força SQLite (desenvolvimento local)
ATLAS_CASHFLOW_DB=supabase	Força Supabase (produção)
Sem variável	Auto-detect: Supabase se NEXT_PUBLIC_SUPABASE_URL + ANON_KEY existirem

Ambos os backends são **non-destructive**: toda criação de tabela usa `IF NOT EXISTS`. Cada repositório existe em duas implementações em `lib/repositories/sqlite/` e `lib/repositories/supabase/`, ambas implementando a mesma interface TypeScript.

3.4 Tabelas do Banco (17)

Grupo	Tabelas	Propósito
Base	Client, Invoice, Expense, Partner, PartnerTransaction, AITokenLog	Núcleo financeiro
Enterprise	client_accounts, contracts, plans, user_entitlements	Clientes corporativos e planos
FinOps	usage_events, model_rate_cards, search_rate_cards	Rastreamento de custos de IA
Billing	plus_subscriptions, billing_events, invoice_line_items	Assinaturas e split de receita
Pesquisa	research_jobs	ROI de pesquisa
Sistema	system_users, audit_log	RBAC e auditoria

A tabela `usage_events` é o centro do FinOps — cada linha é um evento de uso de IA com custo breakdown completo (tokens, cache, modelo, usuário, custo USD/BRL).

3.5 API Routes

Endpoint	Método	Função
/api/atlas	GET / POST	REST API inbound para L2 Atlas
/api/mcp	GET / POST / DELETE	Transport MCP com 7 ferramentas
/api/webhooks/tokens	POST	Receiver de eventos de uso de IA
/api/engine/evaluate	POST	Motor de degradação ativa
/api/tokens	GET / POST	Log de tokens de IA (legacy)

3.6 Integração com L2 ATLAS

O Cashflow não é isolado — ele conversa com o ATLAS por dois canais complementares:

Canal	Como funciona	Uso
MCP (Model Context Protocol)	ATLAS chama ferramentas via HTTP+JSON tipado com Zod	Queries: listar clientes, resumo financeiro, uso de IA
Webhooks	Cashflow dispara eventos push para o ATLAS	Notificações: fatura paga, orçamento excedido, usuário degradado

4. O QUE VAMOS FAZER (Roadmap)

A visão de longo prazo é evoluir o Cashflow de uma ferramenta interna de FinOps para uma **plataforma financeira universal e modular**, capaz de atender qualquer tipo de negócio brasileiro — do MEI emitindo sua primeira NFS-e à empresa multi-entidade consolidando demonstrativos IFRS.

O master plan completo está em `research/master-plan/MASTER-PLAN.md`. Abaixo, a síntese das 7 fases organizadas por ordem de dependência.

4.1 Fase 1 — Fundações

Antes de qualquer módulo novo, o sistema atual precisa de fundamentos que hoje estão fracos ou inexistentes:

- **Auth + RBAC** — sistema de autenticação e permissões por função (admin, contador, visualizador, AP clerk, AR clerk)
- **Chart of Accounts** — plano de contas estruturado para contabilidade
- **General Ledger** — livro razão com lançamentos em partidas dobradas (débito = crédito). É a peça mais crítica: 28 módulos dependem dela
- **Multi-Tenancy** — isolamento por tenant via PostgreSQL Row Level Security. O `client_accounts` já é a entidade de tenant
- **Plugin System** — esqueleto para módulos carregáveis dinamicamente com manifesto TOML
- **Event Sourcing (prototype)** — journal append-only como trilha de auditoria no GL

4.2 Fase 2 — Módulos Core

- **Accounts Payable** — contas a pagar com state machine (draft → finalized → sent → paid → voided)
- **Accounts Receivable** — contas a receber com os mesmos estados
- **Invoicing** — geração de notas fiscais com itens de linha detalhados
- **Fiscal Year** — gestão de exercícios com travas de período
- **Expenses** — reescrita sobre o GL com categorização avançada e fluxo de aprovação
- **Payments (Initial)** — execução de pagamentos via Pix, boleto

4.3 Fase 3 — Compliance Brasil

Esta é a fase mais longa e de maior valor estratégico. É onde o Cashflow se diferencia de qualquer sistema genérico — compliance profundo com o ecossistema fiscal brasileiro.

- **Tax Engine** — motor de impostos com regras versionadas por data de vigência (IRPJ, CSLL, INSS, Simples Nacional, PIS/COFINS)
- **NFS-e (São Paulo primeiro)** — integração com municípios via padrão ABRASF 2.03
- **NFe** — integração com SEFAZ (SVRS, expandindo para demais estados)
- **SPED** — geração de EFD-Contribuições, EFD-ICMS/IPI, ECD, ECF
- **eSocial** — S-1200 (remuneração) + S-1299 (fechamento)
- **LGPD** — consentimento, retenção programada, erasure

4.4 Fase 4 — Pagamentos e Reconciliação

- **Bank Reconciliation** — matching automático de transações com regras de confiança (60% valor + 25% data + 15% descrição)
- **Payment Gateways** — Asaas, PagSeguro, Mercado Pago, Stripe
- **Banking Integration** — via Belvo para agregação de contas
- **CNAB 240/400 + OFX** — importação de extratos bancários
- **Recurring Billing** — faturamento recorrente automático

4.5 Fase 5 — Enterprise e Multi-Entidade

- **Multi-Entity** — consolidação de múltiplas empresas com intercompany reconciliation
- **Analytics/BI** — dashboards operacionais e comerciais (DuckDB embedded + Metabase SDK)
- **Security Hardening** — envelope encryption AES-256-GCM, PCI SAQ A-EP, pentest
- **Performance** — otimização de queries, materialized views, PgBouncer

4.6 Fase 6 — Crescimento e Marketplace

- **Marketplace SDK** — sistema de plugins público para terceiros
- **Seed plugins (15-20)** — primeira leva de plugins próprios
- **PLG Engine** — invoice-as-viral-vector (cada fatura emitida é um canal de aquisição)
- **Onboarding Wizard** — wizard de 5 etapas para novos tenants em menos de 5 minutos
- **Pricing** — Free / R\$79 / R\$199 / R\$499 + módulos adicionais

4.7 Fase 7 — Escala e Otimização

- **Full Test Suite** — suite completa de testes (ver Seção 5)
- **Load Testing** — 100K entradas de journal, 10 tenants simultâneos
- **Multi-Currency** — suporte a múltiplas moedas com fontes de câmbio (BCB, ECB, Fed)
- **Rust Cementation** — componentes críticos migrados para Rust (decisão D-022)

4.8 Critical Path

A cadeia de dependências crítica que determina a ordem de construção:

Etapa	Módulos	Nível de Dependência
1	Auth → General Ledger	Fundação (0 dependências externas)
2	GL → Tax Engine	Nível 1 (depende do GL)
3	Tax Engine → NFe / SPED	Nível 2 (depende do Tax Engine)
4	NFe/SPED → Multi-Entity	Nível 3
5	Multi-Entity → Marketplace	Nível 4 (terminal)

42 módulos no total, sendo 10 de complexidade XL. O **General Ledger** é o gargalo #1: 28 módulos dependem dele. Se ele estiver errado, 70% do sistema está errado.

5. ESTRATÉGIA DE TESTES

Zero testes existem hoje. Este é o maior risco do projeto. Todo o código está em produção sem nenhuma cobertura automatizada. A estratégia completa está em `research/master-plan/batch4/B4-testing-strategy.md`.

5.1 Pirâmide de Testes

Camada	Cobertura	Ferramenta	O que cobre
Unit (80%)	>90% em lib/	Vitest + fast-check	Cálculos financeiros, invariantes, bordas
Integration (15%)	>80% rotas + repos	Vitest + SQLite in-memory	Wiring entre módulos, API contracts
E2E (5%)	100% fluxos críticos	Playwright	Caminhos felizes de cada domínio

5.2 Prioridade de Implementação

Prioridade	Arquivo	Justificativa
P0	lib/tax.ts	Cálculos de MEI com implicação legal
P0	lib/engine/normalizer.ts	Custo de IA por modelo — impacto financeiro direto
P0	lib/engine/degradation.ts	Lógica de degradação — previne custos explosivos
P1	lib/forecast.ts	Projeções de fluxo de caixa usadas em decisões de negócio
P1	lib/webhooks/dispatcher.ts	Payload shape e comportamento de retry

5.3 Property-Based Testing

Módulos financeiros usarão `fast-check` para verificar invariantes automaticamente em vez de depender apenas de exemplos fixos. Exemplos de propriedades:

Função	Propriedade
<code>calculateMEITax</code>	<code>percentUsed</code> sempre em <code>[0, 100]</code>
<code>calculateMEITax</code>	<code>remaining</code> sempre ≥ 0
<code>calculateUsageCost</code>	$\text{costBrl} === \text{costUsd} \times \text{fxRate}$
<code>calculateUsageCost</code>	todos os custos ≥ 0
<code>generateCashFlowProjection</code>	<code>cumulativeBalance</code> é soma cumulativa de <code>estimatedProfit</code>

5.4 Golden Master (Snapshot) Tests

Relatórios financeiros e outputs de forecast usarão golden masters — snapshots do output esperado que falham em qualquer drift não intencional:

- Mudança intencional → executar com `UPDATE_GOLDEN_MASTERS=1 vitest` para regenerar

- Commit do snapshot atualizado + justificativa no PR
- CI sem o flag → falha se output mudou inesperadamente

5.5 Contract Tests (Dual-Backend)

Cada implementação de repositório (SQLite e Supabase) roda os mesmos testes de contrato, garantindo que ambas as versões se comportam identicamente para quem as consome.

- Round-trip CRUD: create → getById → update → getById → delete
- Filtros: getByStatus, getByMonth, getClient aplicam corretamente
- Foreign keys: cascade e set null funcionam como esperado

5.6 Fluxos E2E (Playwright)

Fluxo	Etapas
Fatura → pagamento → reconciliação	Criar cliente → criar fatura → marcar paga → verificar status
Despesa recorrente → projeção	Criar despesa recorrente → verificar projeção mensal
Carteira de sócio	Injeção → retirada → verificar saldo
Degradação ativa	Simular uso → exceder hard cap → verificar webhook
Forecast	Input clientes + despesas → verificar projeção de 6 meses

5.7 Refactors Necessários Antes de Testar

O código atual tem dependências diretas que impedem testes isolados. Quatro refactors pequenos e não-breaking são necessários:

Arquivo	Mudança	Por quê
lib/db/index.ts	Aceitar Database injetado em vez de singleton global	SQLite in-memory nos testes
lib/engine/normalizer.ts	Aceitar Supabase client como parâmetro	Mockar na unit test
lib/engine/degradation.ts	Aceitar client + dispatcher como parâmetros	Mockar na unit test
lib/webhooks/dispatcher.ts	Aceitar fetch como parâmetro ou usar vi.stubGlobal	Mockar requisições HTTP

5.8 Pipeline CI/CD de Testes

Os testes serão executados em pipeline GitHub Actions com gates progressivos:

- Unit → Integration → E2E (gates sequenciais, E2E só roda se unit passar)
- Cobertura mínima: 85% statements, 80% branches, 85% functions — PRs abaixo são bloqueados
- Security scan (ZAP Baseline) toda noite em staging

Estimativa de Esforço — Testes

Atividade	Horas
Setup (vite + Playwright + CI pipeline)	28h
Unit tests — arquivos críticos (tax, normalizer, degradation)	~20h
Integration tests — repositories + API routes	~20h
E2E tests — 6 fluxos críticos	~16h
Contract tests + golden masters	~10h
Total — primeira rodada	~94h
Manutenção por sprint	~9h

6. DOCUMENTOS RELACIONADOS

Documento	Localização	Conteúdo
Master Plan	research/master-plan/MASTER-PLAN.md	Roadmap completo, 42 módulos, decisões arquiteturais, riscos
Relatório Técnico	docs/compose/reports/cashflow-code-report.md	Análise detalhada do código-fonte
Estratégia de Testes	research/master-plan/batch4/B4-testing-strategy.md	Plano completo de testes com estimativas
Pesquisa Modular	research/modular-cashflow/REPORT.md	42 descobertas sobre módulos financeiros universais
Schema SQL	supabase/schema.sql	DDL completo + 6 RPC functions PostgreSQL

L2 SYSTEMS — Todos os direitos reservados.

Este documento é confidencial e de uso interno da equipe L2 Systems e colaboradores autorizados. Não deve ser distribuído sem autorização.